

DECLARATIVE PIPELINE KEYWORDS (Jenkinsfile)

<code>pipeline {}</code>	Root block — wraps the entire pipeline	<i>Required in every Jenkinsfile</i>
<code>agent any</code>	Run on any available Jenkins agent/node	<i>Or: <code>agent { label "linux" }</code></i>
<code>agent none</code>	No default agent — set per stage	<i>Define agent inside each stage</i>
<code>agent { docker {} }</code>	Run stage inside a Docker container	<i><code>agent { docker { image "node:18" } }</code></i>
<code>stages {}</code>	Container for all your stage blocks	
<code>stage("Name") {}</code>	A named phase of your pipeline	<i>e.g. Build, Test, Deploy</i>
<code>steps {}</code>	The actual commands inside a stage	<i>Goes inside stage {}</i>
<code>post {}</code>	Actions after pipeline/stage completes	<i>always, success, failure, cleanup</i>
<code>environment {}</code>	Define env variables for the pipeline	<i><code>environment { ENV = "prod" }</code></i>
<code>parameters {}</code>	Declare user inputs for manual runs	<i>string, choice, booleanParam</i>
<code>triggers {}</code>	Auto-run conditions (cron, SCM poll)	<i><code>triggers { cron("H/15 * * * *") }</code></i>
<code>options {}</code>	Pipeline-level settings	<i>timeout, retry, buildDiscarder</i>
<code>when {}</code>	Conditional step/stage execution	<i><code>when { branch "main" }</code></i>
<code>parallel {}</code>	Run multiple stages simultaneously	<i>Wrap stages inside</i>

BUILT-IN PIPELINE STEPS

<code>sh "command"</code>	Run a shell command	<i><code>sh "npm test"</code></i>
<code>bat "command"</code>	Run a Windows command	<i>Windows agents only</i>
<code>echo "message"</code>	Print a message to the log	
<code>checkout scm</code>	Clone the repo that triggered the build	<i>Auto-configured by Jenkins</i>
<code>git url: "..."</code>	Clone a specific Git repo	
<code>withCredentials([...]) {}</code>	Safely inject secrets into the block	<i>Never echo a credential!</i>
<code>credentials("id")</code>	Reference a stored credential by ID	<i>usernamePassword, string, file</i>
<code>archiveArtifacts "**/*.jar"</code>	Save build artifacts in Jenkins	<i>Download from build page</i>
<code>junit "results/*.xml"</code>	Publish JUnit test results	<i>Shows pass/fail graphs</i>
<code>stash name:"...", ...</code>	Save files to pass between stages	<i>Use with agent none</i>
<code>unstash "name"</code>	Retrieve stashed files	
<code>input "Deploy to prod?"</code>	Pause and wait for human approval	<i>Adds a button in the UI</i>
<code>sleep 30</code>	Pause for N seconds	
<code>error "message"</code>	Fail the build with a message	
<code>catchError {}</code>	Continue pipeline even if block fails	

PLUGINS & SHARED LIBRARIES

<code>@Library("name") _</code>	Import a shared library at top of file	<i>_ is required after the import</i>
<code>pipeline-utility-steps</code>	readJSON, readYaml, findFiles, zip...	<i>Very useful plugin</i>
<code>kubernetes plugin</code>	Spin up pods as Jenkins agents	<i>Cloud-native CI</i>
<code>Google Cloud Build plugin</code>	Trigger GCB from Jenkins	
<code>docker-workflow plugin</code>	Enables <code>agent { docker {} }</code> blocks	
<code>credentials plugin</code>	Manage secrets in Jenkins UI	<i>Bind with <code>withCredentials</code></i>
<code>Blue Ocean</code>	Modern visual pipeline UI	<i>Optional but beginner-friendly</i>

JENKINS + GCP PATTERNS

gcloud auth activate...	Authenticate to GCP in a step	<i>Use a service account key file</i>
withCredentials file	Inject GCP key JSON as a temp file	<i>Safest way to use SA keys</i>
sh "gcloud builds submit"	Trigger a Cloud Build from Jenkins	
sh "docker push gcr.io..."	Push to GCR after docker build	<i>Auth with gcloud first</i>
sh "kubectl apply -f ."	Deploy to GKE from Jenkins	<i>Need kubeconfig credential</i>
sh "terraform apply"	Run Terraform from a Jenkins stage	<i>Store state in GCS</i>
Artifact Registry	Push images to AR instead of GCR	<i>Preferred modern option</i>

MINIMAL JENKINSFILE EXAMPLE

```
pipeline { agent { docker { image "node:20-alpine" } } environment { GCP_PROJECT = "my-gcp-project" } stages { stage("Install") { steps { sh "npm ci" } } stage("Test") { steps { sh "npm test" junit "test-results/*.xml" } } stage("Deploy") { when { branch "main" } steps { input "Deploy to production?" sh "gcloud app deploy --project=${GCP_PROJECT}" } } } post { failure { echo "Build failed – check the logs above" } success { echo "All good!" } } }
```

Tip: Store your Jenkinsfile in the root of your repo — Jenkins reads it automatically on each push.

jenkins.io/doc